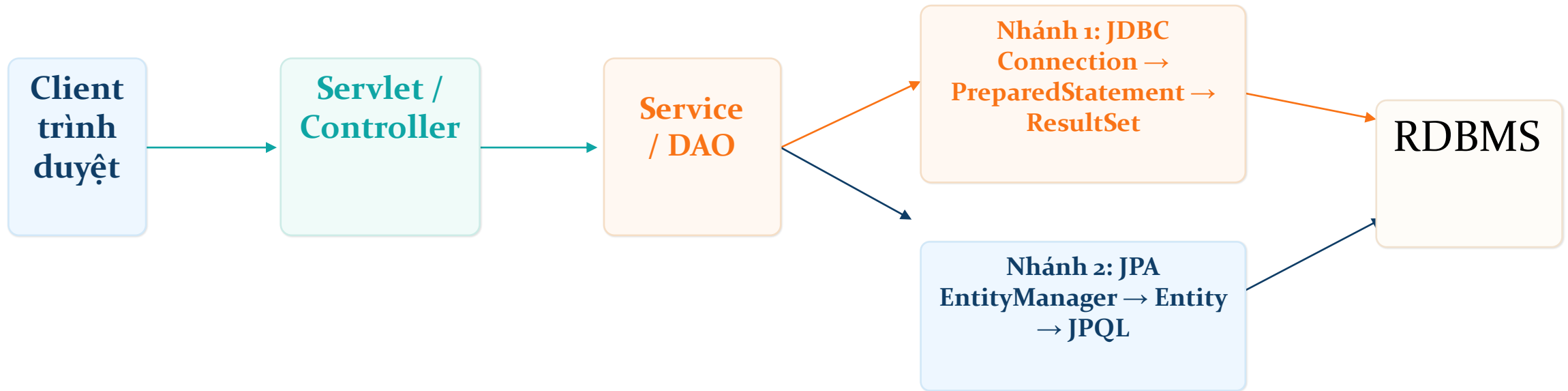


Bài 1



JDBC & JPA

JDBC và JPA trong ứng dụng Web



- Điểm chung: đều phục vụ tầng truy cập dữ liệu, nhưng mức trừu tượng khác nhau
- Điểm cần nhớ: JDBC và JPA không thay thế database; chúng là hai cách khác nhau để ứng dụng Java truy cập RDBMS.

Giới thiệu JDBC

- JDBC (Java Database Connectivity) là API chuẩn để Java truy cập dữ liệu quan hệ.
- Lập trình viên làm việc trực tiếp với SQL và các đối tượng như Connection, PreparedStatement, ResultSet.
- JDBC phù hợp khi cần kiểm soát chặt chẽ câu lệnh SQL, transaction và tối ưu hiệu năng ở mức thấp.
- Đây là nền tảng mà nhiều thư viện/ORM khác vẫn dựa vào ở tầng dưới.

1 Nạp driver / cấu hình DataSource

2 Mở kết nối Connection

3 Tạo lệnh SQL PreparedStatement

4 Thực thi executeQuery / update

5 Đọc dữ liệu ResultSet

6 Đóng tài nguyên close()

Mô hình tư duy: “Viết SQL → gửi cho DB → nhận kết quả → tự map sang object”

Ví dụ code JDBC trong Servlet/DAO

DAO sử dụng JDBC

```
1 String sql = "SELECT id, name, email FROM students";
2 try (Connection con = ds.getConnection();
3     PreparedStatement ps = con.prepareStatement(sql);
4     ResultSet rs = ps.executeQuery()) {
5
6     List<Student> list = new ArrayList<>();
7     while (rs.next()) {
8         list.add(new Student(
9             rs.getInt("id"),
10            rs.getString("name"),
11            rs.getString("email")
12        ));
13    }
14    return list;
15 }
```

Slide demo

- SQL nằm rõ ràng trong code.
- Tự map từng cột sang đối tượng Student.
- Đóng tài nguyên tự động bằng try-with-resources.

Điểm quan sát

- Ngắn gọn với truy vấn đơn giản.
- Dễ tối ưu SQL theo ý muốn.
- Khi nghiệp vụ lớn, đoạn map thủ công sẽ lặp lại nhiều.

Ưu và nhược điểm của JDBC

■ Ưu điểm:

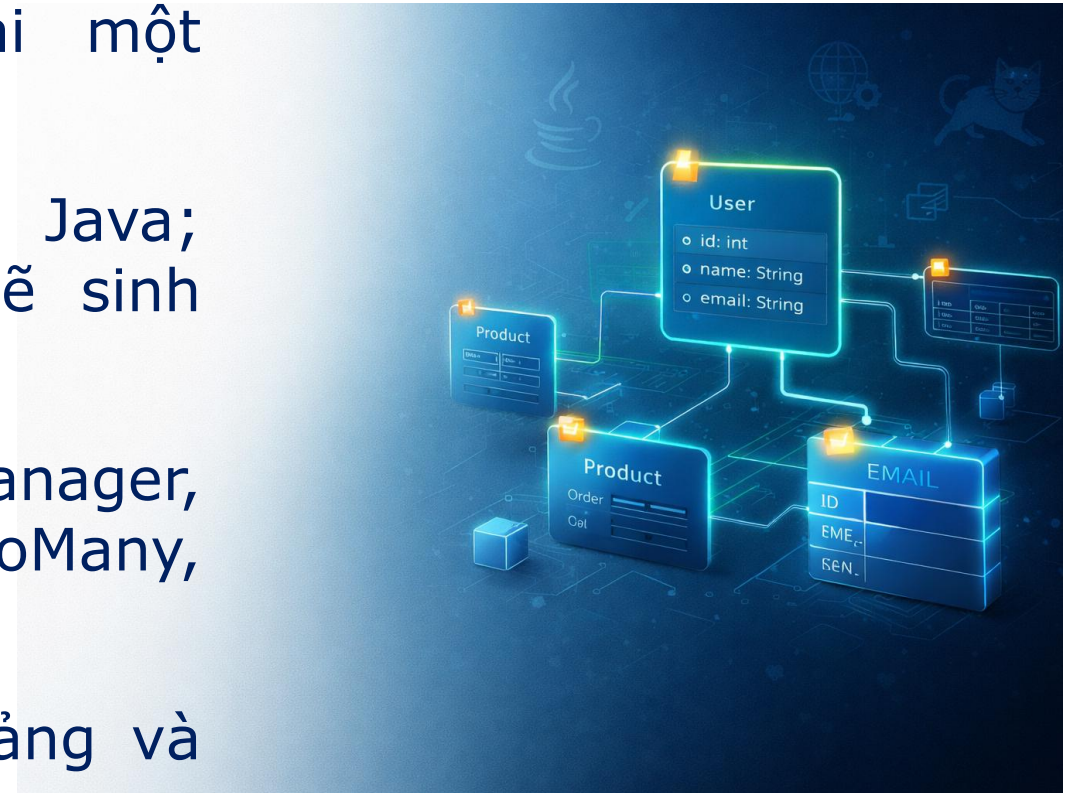
- Kiểm soát SQL chi tiết: join, hint, stored procedure, transaction...
- Dễ tối ưu cho truy vấn phức tạp hoặc khối lượng dữ liệu lớn.
- Hiểu sâu cách database hoạt động vì lập trình viên nhìn thấy SQL thật.
- Ít lớp trừu tượng, ít “bí ẩn” khi debug hoặc đo hiệu năng.

■ Nhược điểm:

- Code lặp lại nhiều: mở kết nối, bind tham số, map dữ liệu, đóng tài nguyên.
- Khó bảo trì khi số lượng bảng, quan hệ và truy vấn tăng mạnh.
- Phụ thuộc nhiều vào SQL nên việc đổi DB hoặc đổi cấu trúc bảng tốn công hơn.
- Nếu viết cầu thả dễ phát sinh lỗi tài nguyên, lỗi transaction, SQL injection.

Giới thiệu JPA

- JPA là đặc tả (spec), không phải một framework cụ thể.
- Lập trình viên làm việc với entity Java; provider như Hibernate/EclipseLink sẽ sinh SQL tương ứng.
- Khái niệm trung tâm: Entity, EntityManager, persistence context, quan hệ @OneToMany, @ManyToOne...
- JPA giúp giảm map thủ công giữa bảng và object trong ứng dụng Java.



Ví dụ code JPA

Entity + thao tác

persist/find

```
1 @Entity
2 public class Student {
3     @Id @GeneratedValue
4     private Integer id;
5     private String name;
6     private String email;
7 }
8
9 Student s = new Student();
10 s.setName("An");
11 s.setEmail("an@demo.vn");
12 em.persist(s);
```

Điểm khác JDBC

- Không tự map từng cột khi persist entity.
-
- EntityManager quản lý trạng thái object.
-
- SQL thường được provider sinh ở tầng dưới.

Ưu và nhược điểm của JPA

■ Ưu điểm:

- Giảm đáng kể code CRUD lặp lại và phần map ResultSet thủ công.
- Thiết kế gần với tư duy OOP: entity, quan hệ, vòng đời đối tượng.
- Dễ mở rộng với dự án có nhiều bảng, nhiều quan hệ và nghiệp vụ CRUD.
- Tăng tốc độ phát triển khi dùng repository/service theo kiến trúc rõ ràng.

■ Nhược điểm:

- Khó thấy ngay SQL thật nếu sinh viên chưa bật log hoặc chưa hiểu provider.
- Có thể phát sinh truy vấn không tối ưu nếu dùng quan hệ/lazy/eager chưa đúng.
- Tốn thời gian học thêm annotation, lifecycle, cache, transaction context.
- Một số báo cáo/truy vấn đặc thù vẫn cần native SQL hoặc JDBC.

So sánh JPA và JDBC

Tiêu chí	JDBC	JPA
Mức trừu tượng	Thấp, gần SQL	Cao hơn, gần object/entity
Tốc độ phát triển CRUD	Chậm hơn	Nhanh hơn
Kiểm soát truy vấn	Rất cao	Trung bình đến cao, tùy provider/native query
Map bảng ↔ object	Thủ công	Tự động hơn qua annotation/mapping
Học bản chất DB/SQL	Rất tốt	Tốt nếu vẫn xem log SQL
Phù hợp nhất	Báo cáo phức tạp, tối ưu sâu, SQL đặc thù	Ứng dụng CRUD nhiều, domain model rõ ràng
Thực tế dự án	Có thể dùng riêng hoặc làm nền	Thường kết hợp thêm native query/JDBC khi cần

Kết luận thực tế: nhiều hệ thống dùng JPA cho phần lớn nghiệp vụ và dùng JDBC/native SQL cho các truy vấn đặc biệt.

Tổng kết

- JDBC = API chuẩn, làm việc gần SQL và kiểm soát sâu.
- JPA = đặc tả ORM, làm việc gần object/entity hơn.
- JDBC mạnh khi tối ưu truy vấn hoặc cần SQL đặc thù.
- JPA mạnh khi ứng dụng có nhiều CRUD và mô hình domain rõ.
- Dùng JPA vẫn phải hiểu SQL, transaction và database.
- Trong thực tế, giải pháp kết hợp thường rất phổ biến.

Thông điệp:

- Học JDBC để hiểu bản chất.
- Học JPA để tăng năng suất.
- Biết so sánh để chọn đúng công cụ.